DMA na ESP32-S3

Direct Memory Access com PlatformIO

Arquitetura | Fluxo de Dados | Leitura Analógica | Aplicações Práticas

Engenharia de Sistemas Embarcados

01 — Fundamentos de DMA

O que é Direct Memory Access e por que é essencial

O que é DMA?

Direct Memory Access (DMA)

Mecanismo de hardware que permite transferência de dados entre periféricos e memória sem intervenção da CPU.

Conceito Central

- Controlador dedicado no barramento do sistema
- Opera em paralelo com a CPU
- Gerencia transferências bloco a bloco automaticamente
- Sinaliza conclusão via interrupção (IRQ)

Analogia

Imagine um gerente (CPU) que delega o transporte de caixas a um assistente (DMA), ficando livre para tomar decisões estratégicas.

CPU-Driven vs DMA — Comparação Visual

❌ Acesso Tradicional (CPU-Driven)

Periférico (ADC/SPI) → CPU (Ocupada!) → Memória (RAM)

CPU 100% Bloqueada!

✅ Acesso com DMA

Periférico (ADC/SPI) → DMA Controller → Memória (RAM)

CPU Livre! Outras tarefas

Com DMA, a CPU fica livre para executar código enquanto os dados são transferidos automaticamente pelo hardware

Por que DMA é Essencial?

Sistemas Embarcados Modernos

- Múltiplos periféricos operando simultaneamente
- Taxas de aquisição de dados cada vez maiores
- Requisitos de tempo real com latências críticas

Sem DMA — Problemas

- CPU gasta ciclos movendo bytes (até **70-90%** do tempo)
- Perda de amostras em alta frequência
- Jitter temporal inaceitável para controle PID

Com DMA — Solução

- Transferências automáticas em background
- Zero perda de amostras com buffer circular
- Determinismo temporal para aplicações críticas

02 — Arquitetura ESP32-S3

Organização interna, barramentos e controlador DMA

Arquitetura Interna — Visão Geral

Controlador GDMA — Detalhes

General DMA (GDMA)

- 5 canais de transmissão (TX) + 5 canais de recepção (RX)
- Linked-list descriptors para transferências encadeadas
- Suporte a burst transfers (4-byte aligned)
- Prioridade configurável por canal

Modos de Transferência

- **Periférico → Memória** (leitura ADC, recepção SPI)
- **Memória → Periférico** (transmissão SPI, saída I2S)
- **Memória → Memória** (cópia rápida de blocos)

Interrupções GDMA

- Transfer complete, Half-transfer, Error
- Permite processamento em double-buffer (ping-pong)

Integração DMA + Periféricos

ADC + DMA

- Modo contínuo com DMA
- Buffer circular automático
- Até 2 MSPS por canal

SPI + DMA

- Transferências full-duplex
- Até 80 MHz clock
- Ideal para displays e sensores

I2S + DMA

- Streaming de áudio contínuo
- Buffer double-ping-pong
- Até 96 kHz / 24-bit

UART + DMA

- Recepção sem polling
- Ideal para GPS, Modbus
- Zero overhead na CPU

03 — Fluxo de Dados

Convencional vs DMA — Linha do tempo de execução

Fluxo Convencional (sem DMA)

⚠ **CPU 100% ocupada —** Cada byte passa pela CPU. Não há paralelismo. Gargalo em alta frequência.

Fluxo com DMA — Execução Paralela

CPU	**Processa dados**	**Algoritmo PID**	**Comunicação WiFi**	
DMA	**ADC → RAM**	**ADC → RAM**	**ADC → RAM**	**ADC → RAM**
Barramento	DMA	DMA	DMA	DMA
Periférico	Aquisição contínua (sem interrupção da CPU)			

tempo →

✅ **CPU livre para processar —** DMA move dados em paralelo. Aquisição contínua sem perda de amostras.

Gargalos e Latência — Análise Técnica

Sem DMA — Gargalos

- Cada leitura ADC: ~**20-50 ciclos** de CPU por amostra
- A 1 MSPS = **20-50M ciclos/s** só para I/O
- CPU @ 240 MHz → **~20% gasto só em transferência**
- Jitter de interrupção: **±1-5 µs**

Com DMA — Desempenho

- CPU overhead: **~0%** durante transferência
- Throughput máximo do barramento utilizado
- Jitter de aquisição: **±0 (hardware-timed)**
- Buffer circular elimina perda de dados

04 — Vantagens do DMA

Performance, determinismo e aplicações em tempo real

4 Pilares do DMA

Redução de Carga

CPU livre para lógica de aplicação

De ~90% para ~0% de uso em I/O

Maior Throughput

Velocidade máxima do barramento

Transferências burst otimizadas

Determinismo

Timing preciso por hardware

Jitter ±0 na aquisição

Tempo Real

Ideal para controle e monitoramento

Aquisição contínua garantida

Exemplos de Uso do DMA

Aquisição Contínua de Sinais

- Sensores de vibração, temperatura, pressão
- ADC contínuo a 100 kHz+ sem perda

Processamento de Áudio

- I2S + DMA para streaming em tempo real
- Microfone MEMS → FFT → classificação ML

Comunicação de Alta Velocidade

- SPI Master a 80 MHz com DMA full-duplex
- Displays TFT, memória externa, comunicação inter-chip

Controle Industrial

- Malhas de controle PID com aquisição determinística

05 — DMA + Leitura Analógica

ADC contínuo, buffer circular e alta frequência

ADC + DMA na ESP32-S3

Leitura Simples (oneshot)

- CPU solicita → ADC converte → CPU lê resultado
- Bloqueante: CPU espera ~**25 µs** por conversão
- Máximo prático: ~40 kSPS

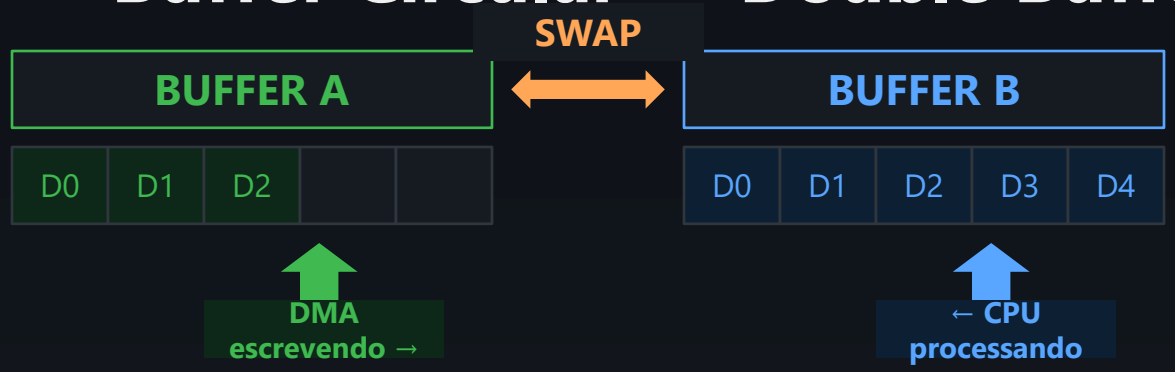
Leitura Contínua com DMA

- ADC opera em modo contínuo (free-running)
- DMA transfere automaticamente para buffer na RAM
- Interrupção ao preencher metade do buffer (half-transfer)
- CPU processa enquanto DMA preenche outra metade

Especificações Técnicas (ESP32-S3)

- 2x ADC SAR, 12-bit, até 20 canais
- Taxa máxima com DMA: **~83.3 kHz por canal (ESP-IDF)**

Buffer Circular — Double Buffering



Como funciona

1. DMA preenche Buffer A com dados do ADC
2. Quando A está cheio, IRQ sinaliza
3. DMA troca para Buffer B (swap)
4. CPU processa Buffer A enquanto DMA enche B
5. Ciclo contínuo sem perda de dados

Vantagem: Zero gap entre aquisição e processamento

O padrão ping-pong é implementado automaticamente pelo driver adc_continuous do ESP-IDF

06 — Configuração PlatformIO

Ambiente, framework e configuração do projeto

platformio.ini — Configuração Base

```
; platformio.ini
[env:esp32s3]
platform = espressif32
board = esp32-s3-devkitc-1
framework = espidf

; Configurações de build
monitor_speed = 115200
board_build.flash_mode = dio
board_build.f_cpu = 240000000L
```

Requisitos do Ambiente

- **Framework: ESP-IDF** (obrigatório para DMA)
- Arduino framework não expõe driver adc_continuous

Bibliotecas Necessárias

- `esp_adc/adc_continuous.h`
- `esp_log.h`
- `freertos/FreeRTOS.h`

Nota Importante

O driver adc_continuous gerencia automaticamente o GDMA. Não é necessário configurar DMA manualmente.

07 — Código Prático

Leitura ADC contínua com DMA — Exemplo funcional

Código — Configuração do ADC + DMA

```c
#include <string.h>
#include "esp_adc/adc_continuous.h"
#include "esp_log.h"

// Definições do buffer DMA
#define ADC_BUFFER_SIZE  1024
#define ADC_SAMPLE_FREQ  20000  // 20 kHz

static adc_continuous_handle_t adc_handle = NULL;

void adc_dma_init(void) {
    adc_continuous_handle_cfg_t handle_cfg = {
        .max_store_buf_size = 4096,
        .conv_frame_size = ADC_BUFFER_SIZE,
    };
    adc_continuous_new_handle(&handle_cfg, &adc_handle);

    // Configura canal ADC (GPIO4 = ADC1_CH3)
    adc_digi_pattern_config_t pattern = {
        .atten = ADC_ATTEN_DB_12,
        .channel = ADC_CHANNEL_3,
        .unit = ADC_UNIT_1,
        .bit_width = ADC_BITWIDTH_12,
    };
    // ... (continua no próximo slide)
}
```

Código — Callback e Loop Principal

```
// Callback quando DMA completa conversão
static bool IRAM_ATTR adc_conv_done_cb(...) {
    BaseType_t yield = pdFALSE;
    vTaskNotifyGiveFromISR(task_handle, &yield);
    return (yield == pdTRUE);
}

// Loop principal: lê buffer DMA
void app_main(void) {
    adc_dma_init();
    adc_continuous_start(adc_handle);
    uint8_t result[ADC_BUFFER_SIZE] = {0};
    uint32_t ret_num = 0;

    while(1) {
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
        adc_continuous_read(adc_handle,
            result, ADC_BUFFER_SIZE, &ret_num, 0);
        // Processar dados aqui...
    }
}
```

Fluxo de Execução do Código

	1. adc_dma_init()	Aloca handle + configura canal
	2. adc_continuous_start()	Inicia conversão + DMA
	3. ulTaskNotifyTake()	Task bloqueia esperando IRQ
LOOP	4. Callback IRQ (DMA done)	Buffer cheio → notifica task
LOOP	5. adc_continuous_read()	Lê dados do buffer DMA
LOOP	6. Processar dados	FFT, filtro, envio WiFi...

08 — Comparativo Final

Polling vs Interrupt vs DMA — Quando usar cada abordagem

Comparação: Polling vs Interrupt vs DMA

Critério	Polling	Interrupção	DMA
CPU Overhead	Muito alto (~100%)	Médio (~20-40%)	Mínimo (~0%)
Throughput	Baixo	Médio	Máximo
Latência	Variável	Baixa	Determinística
Complexidade	Simples	Média	Maior
Uso de Energia	Alto (CPU ativa)	Moderado	Baixo
Quando usar	Protótipos	Eventos esporádicos	Alta freq. contínua

Obrigado!

DMA no ESP32-S3 — Liberte sua CPU

docs.espressif.com/projects/esp-idf